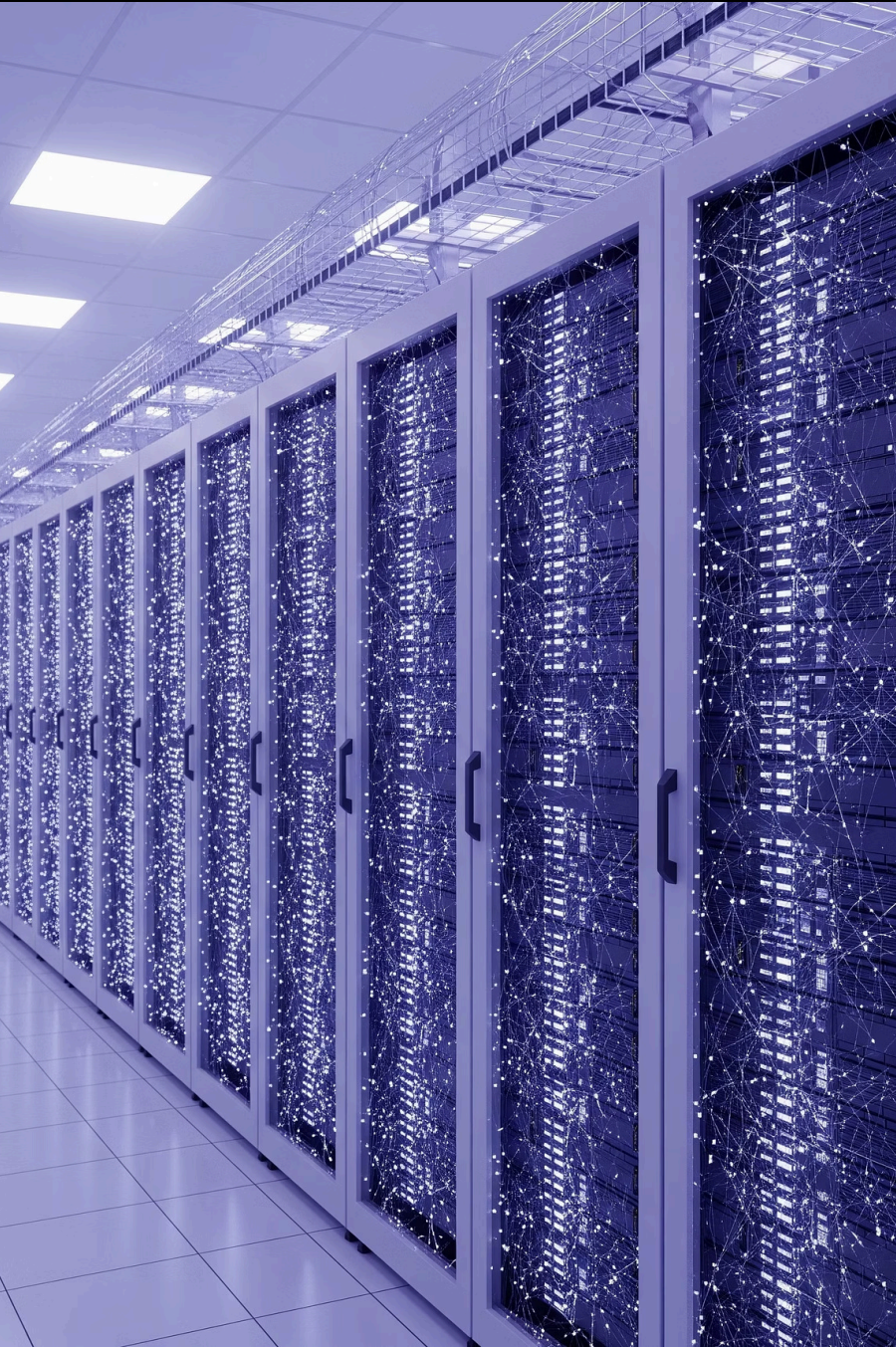




Isolation Strategies in AI Systems

Preventing tenants from interfering with each other, protecting sensitive data & workloads, and ensuring multi-tenant security and reliability

Why Isolation Matters

Multi-tenancy creates shared infrastructure, which introduces significant risks:

Data leakage between tenants

Sensitive information flowing across tenant boundaries

Noisy neighbors hogging GPUs

Performance degradation when one tenant monopolizes shared resources

Security breaches if boundaries weak

Exploitation of insufficiently isolated environments



Isolation requires striking the perfect balance between efficiency and safety in AI infrastructure.



Types of Isolation

1

Resource Isolation

Separating compute, storage, and network resources between tenants

2

Data Isolation

Protecting datasets, feature stores, and model artifacts from unauthorized access

3

Process Isolation

Using containers, VMs, and sandboxes to create secure execution environments

4

Security Isolation

Implementing RBAC, namespaces, and encryption to enforce access boundaries

Resource Isolation

→ Kubernetes Namespaces

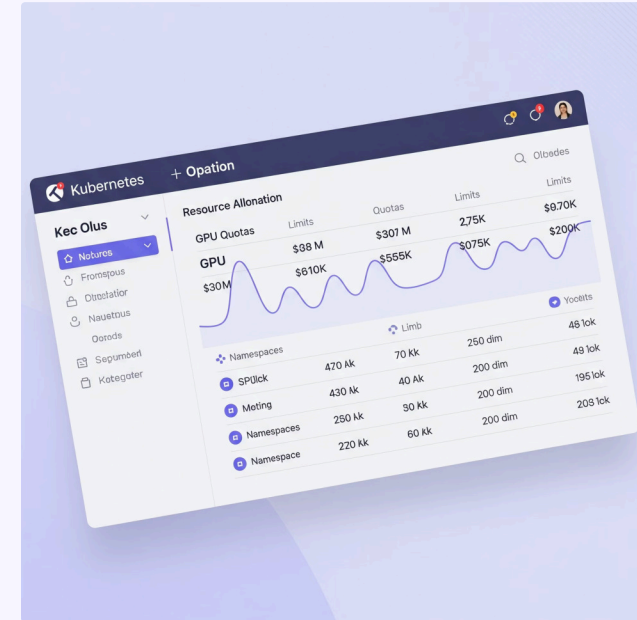
Logical separation of teams/tenants within the same cluster

→ Dedicated Node Pools

Separate high-priority production workloads from research experiments

→ GPU Quotas & Limits

Enforced per namespace to prevent resource starvation



Resource isolation prevents one team from starving another of critical compute resources, especially expensive GPUs and specialized AI accelerators.

Data Isolation

Separate Storage

Dedicated buckets or prefixes per tenant prevent data cross-contamination

RBAC + IAM Policies

Fine-grained access controls at the dataset level ensure proper authorization

Project-Scoped Features

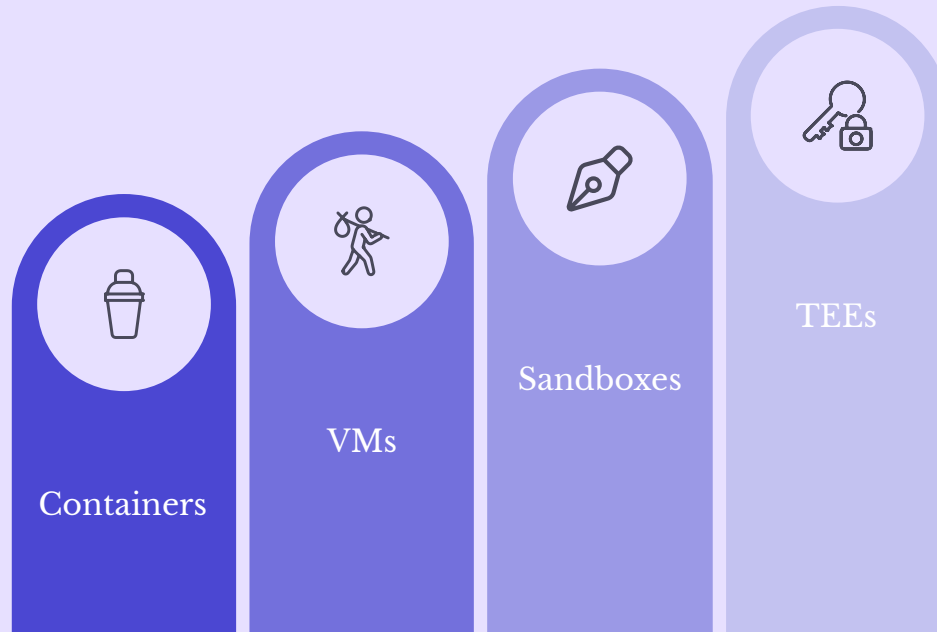
Feature stores with tenant isolation prevent feature leakage between projects

Per-Tenant Encryption Keys

Tenant-specific keys ensure data remains protected even if storage is compromised



Process Isolation



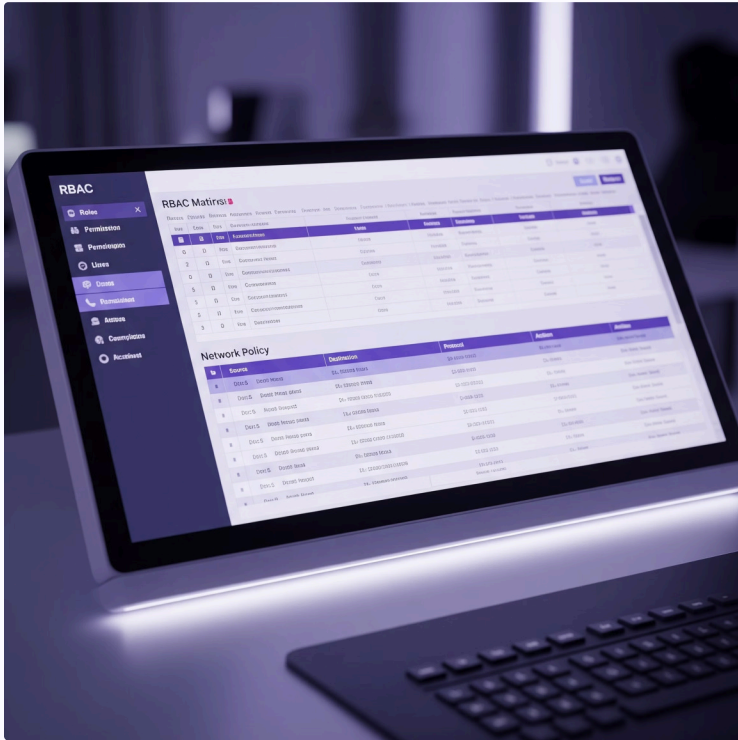
Containers: Lightweight isolation via cgroups & namespaces

VMs: Hardware-level isolation for untrusted tenants

Sandboxes: gVisor, Firecracker for enhanced security

TEEs: Hardware-based isolation for highest security needs

Security Isolation



RBAC & IAM

Identity-based access controls enforce tenant boundaries

Network Policies

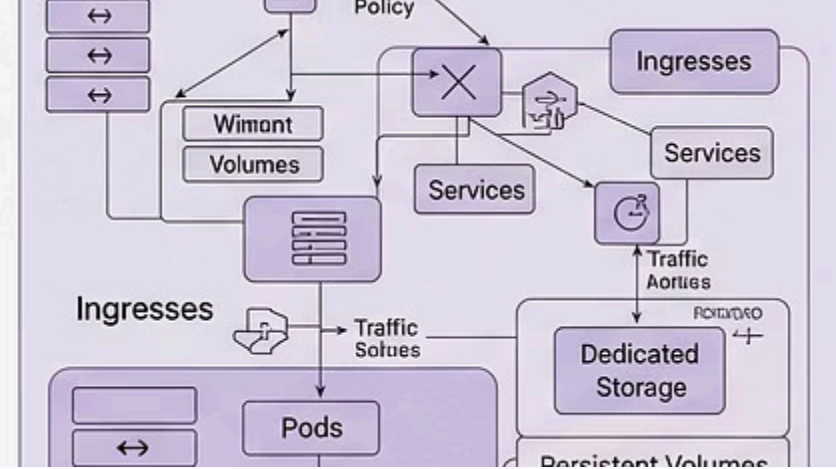
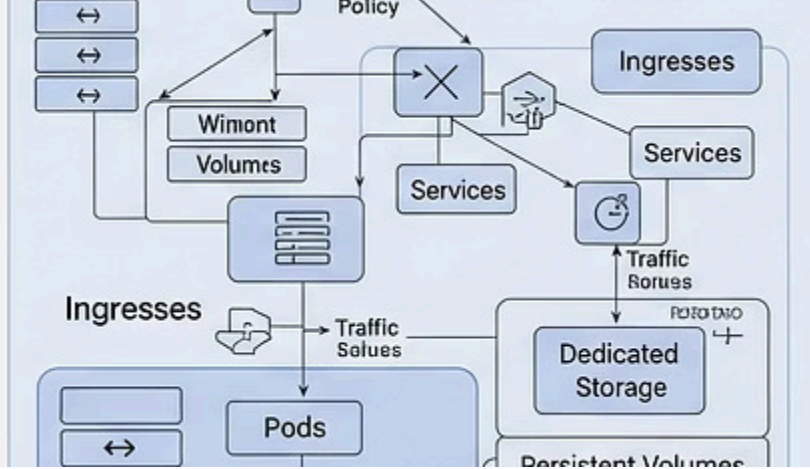
Kubernetes and Istio rules block unauthorized cross-tenant traffic

API Gateways

Per-tenant tokens, rate limits, and request validation

Tenant-Scoped Monitoring

Logging and monitoring segregated by tenant for visibility



Example: Kubernetes Multi-Tenant Isolation

Tenant A

- Dedicated namespace
- GPU quota: 4 GPUs
- Storage: s3://ml-team-a/*
- Network policy: intra-namespace only

Tenant B

- Separate namespace
- GPU quota: 2 GPUs
- Storage: s3://ml-team-b/*
- Network policy: intra-namespace only

RBAC ensures Tenant A cannot access Tenant B's resources, while [Istio enforces network segmentation](#) between namespaces.

Example: Storage Isolation (AWS)



Team A Storage

s3://ml-team-a/* with encryption key A



Separate IAM Roles

Team-specific permissions



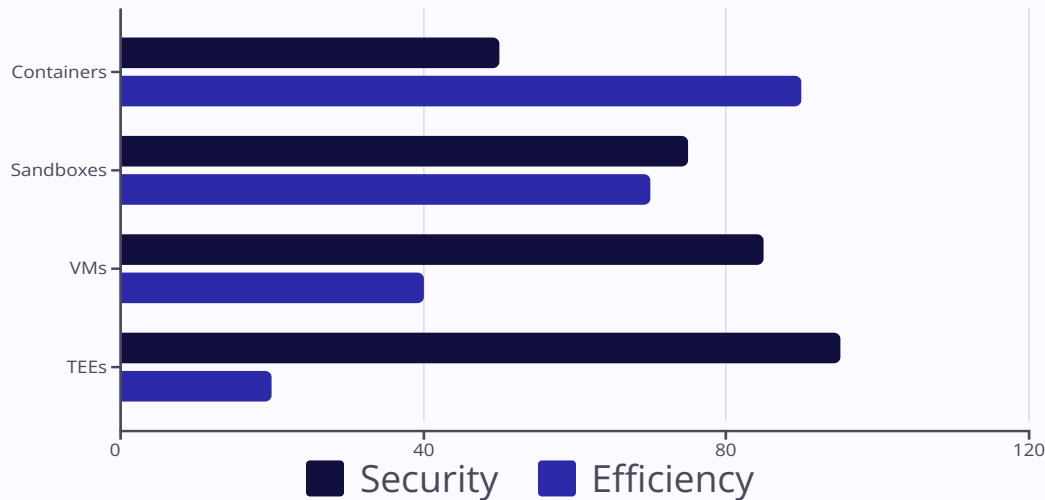
KMS Encryption

Tenant-specific encryption keys



Even with shared infrastructure, tenant data remains fully isolated through IAM policies and encryption.

Tradeoffs in Isolation



Choosing the Right Tradeoff

The appropriate isolation strategy depends on:

- **Security requirements:** Higher sensitivity requires stronger isolation
- **Compliance mandates:** Regulations may dictate minimum isolation levels
- **Workload criticality:** Production vs. research environments
- **Cost constraints:** Stronger isolation generally costs more

Monitoring Isolation Boundaries

Key Metrics to Monitor

- GPU utilization per namespace/tenant
- Cross-namespace network traffic
- Resource quota consumption trends
- Authorization failures at isolation boundaries

Monitoring Stack



Prometheus for metrics collection



Grafana for visualization



Kubecost for resource tracking



Real-World Practices



Google Vertex AI

Project-level isolation with IAM and VPC Service Controls



AWS SageMaker

User workspaces with dedicated compute and S3 bucket isolation



NVIDIA DGX Cloud

Containerized workspaces on shared GPU infrastructure



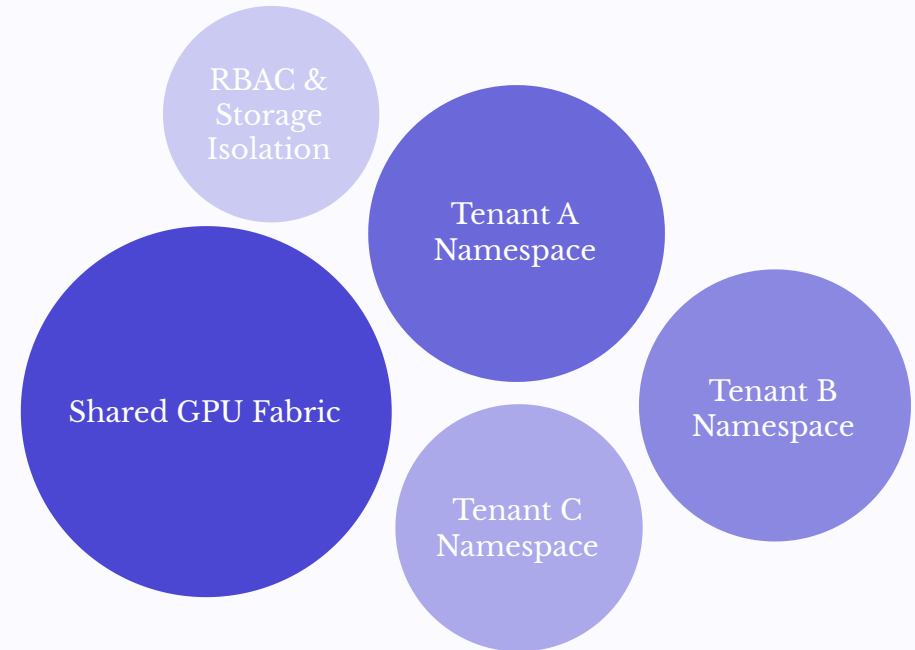
Databricks

Workspace-level isolation for ML clusters and notebooks

Each platform implements tenant isolation differently, but all share common principles: [separate compute environments](#), [isolated storage](#), and [strong access controls](#).



Isolation: The Shield for Multi-Tenant AI



Proper isolation creates a shield that allows multiple tenants to safely coexist on shared infrastructure while maintaining security and performance guarantees.

Summary

→ Multi-Faceted Approach

Isolation strategies must protect data, compute, processes, and security boundaries

→ Toolbox

Leverage namespaces, RBAC, IAM, KMS, and TEEs to create isolation layers

→ Balance

Strong isolation trades efficiency for safety—choose based on workload requirements

→ By Design

Multi-tenant AI infrastructure must **design isolation from day one**



Retrofitting isolation is difficult—build it into your architecture from the beginning.